

APOLLO INSTITUTE OF COMPUTER APPLICATION
BACHELOR OF COMPUTER APPLICATION
SEMESTER 03

Object Oriented Programming with Java (BC03001021)

Lecture Notes

UNIT 2. Building Blocks of the Language (8 HOURS)

Lecture	Topics Covered
Lecture 1	Primitive Data Types, User-Defined Data Types
Lecture 2	Identifiers, Literals, Variables, Constants
Lecture 3	Type Conversion, Casting, Scope, Default Values, Wrapper Classes
Lecture 4	Comments, Garbage Collection
Lecture 5	Arrays of Primitive Data Types, Types of Arrays
Lecture 6	Strings, String Operations, StringBuffer
Lecture 7	Operators, Mathematical Functions
Lecture 8	Decision & Control Statements, Loops, Jump Statements

LECTURE 1: Primitive Data Types and User-Defined Data Types

Primitive Data Types

Primitive data types are the **basic data types provided by Java**. They are used to store simple values such as numbers, characters and true/false values.

Java provides **8 primitive data types**:

Data Type	Description	Example
byte	Small integer value	byte a = 10;
short	Integer value	short a = 100;
int	Integer value	int a = 1000;
long	Large integer value	long a = 10000L;
float	Decimal value	float x = 10.5f;
double	Large/precise decimal value	double x = 20.55;
char	Single character	char ch = 'A';
boolean	True or false	boolean b = true;

Integer Data Types

Integer types store **whole numbers without decimal points**.

```
int age = 20;
```

```
long population = 1000000L;
```

Floating-Point Data Types

They store numbers containing decimal values.

```
float price = 25.5f;
```

```
double salary = 25000.75;
```

Character

char stores a **single character**.

```
char grade = 'A';
```

Character values are written inside **single quotes**.

Boolean

boolean stores only two values:

```
true
```

```
false
```

Example:

```
boolean result = true;
```

User-Defined Data Types

User-defined data types are data types **created by the programmer** according to the requirements of the program.

Examples:

- Class
- Interface
- Enum

Example:

```
class Student {  
    int rollNo;  
    String name;  
}
```

Here, `Student` is a user-defined data type.

Key Point

Primitive data types are provided by Java, whereas user-defined data types are created by the programmer.

LECTURE 2: Identifiers, Literals, Variables and Constants

Identifiers

An **identifier** is the name given to program elements such as:

- Variables
- Methods
- Classes
- Objects

Example:

```
int marks = 80;
```

Here, `marks` is an identifier.

Rules of Identifiers

1. An identifier can contain letters, digits, `_` and `$`.
2. It cannot start with a digit.
3. It cannot contain spaces.
4. It cannot be a Java keyword.
5. Java identifiers are **case-sensitive**.

Example:

```
studentName
```

```
StudentName
```

Both are different identifiers.

Literals

A **literal** is a fixed value written directly in a Java program.

Example:

```
int age = 20;
```

Here, 20 is a literal.

Types of Literals

- Integer literal → 100
- Floating-point literal → 10.5
- Character literal → 'A'
- String literal → "Java"
- Boolean literal → true, false
- Null literal → null

Variables

A variable is a **named memory location used to store data**.

Syntax

```
dataType variableName = value;
```

Example:

```
int age = 20;
```

The value of a variable can be changed.

```
age = 21;
```

Constants

A **constant** is a value that **cannot be changed once it is assigned**.

For example, if we want to store the value of π , we may not want the program to accidentally change it.

We can declare it as a constant.

In Java, constants are generally created using the **final** keyword.

Syntax

```
final dataType CONSTANT_NAME = value;
```

Example:

```
final int MAX_MARKS = 100;
```

Here:

- `final` → tells Java that the value cannot be changed.
- `int` → data type.
- `MAX_MARKS` → constant name.
- `100` → constant value.

Example Program

```
class Demo {  
    public static void main(String[] args) {  
        final int MAX_MARKS = 100;  
        System.out.println(MAX_MARKS);  
    }  
}
```

Output:

100

If we try to change it:

```
MAX_MARKS = 150;
```

Java will give a **compile-time error** because a `final` variable cannot be assigned another value.

Why are Constants Used?

Constants are used when a value should remain **fixed throughout the program**.

Examples:

```
final double PI = 3.14159;
```

```
final int DAYS_IN_WEEK = 7;
```

```
final int MAX_STUDENTS = 60;
```

```
final int MIN_AGE = 18;
```

For example, the number of days in a week is always 7, so it makes sense to store it as a constant.

Naming Convention

Java programmers generally write constant names in **uppercase letters**, with multiple words separated by an underscore `_`.

Examples:

```
final int MAX_SIZE = 100;
```

```
final double PI = 3.14;
```

```
final int MIN_VALUE = 0;
```

```
final String COLLEGE_NAME = "ABC College";
```

This is a **convention**, not a compulsory Java rule.

Constant vs Normal Variable

Normal Variable	Constant
Value can be changed	Value cannot be changed
Usually declared without <code>final</code>	Declared using <code>final</code>
Example: <code>int age = 20;</code>	Example: <code>final int MAX_AGE = 100;</code>
Can be reassigned	Cannot be reassigned

Example:

```
int age = 20;
```

```
age = 25;    // Allowed
```

But:

```
final int MAX_AGE = 100;
```

```
MAX_AGE = 120; // Error
```

Important Point for Exam

Constant: A constant is a fixed value that cannot be changed during program execution. In Java, the `final` keyword is used to declare a constant.

Example:

```
final int DAYS = 7;
```

Once `DAYS` is assigned 7, it cannot be assigned another value.

LECTURE 3: Type Conversion, Casting and Scope

Type Conversion and Type Casting both mean converting a value from one data type to another.

However, there is an important difference between them.

1. Type Conversion

Type conversion is the process of converting one data type into another **automatically by Java**.

It is also called **implicit conversion**.

Example

```
int a = 10;
```

```
double b = a;
```

Here Java automatically converts:

```
int → double
```

```
10 → 10.0
```

No special instruction is required from the programmer.

Example Program

```
class Demo {  
    public static void main(String[] args) {  
        int num = 25;  
        double value = num;  
        System.out.println(value);  
    }  
}
```

Output:

25.0

This is called **automatic type conversion** or **widening conversion**.

2. Type Casting

Type casting is the process of converting one data type into another **manually by the programmer**.

It is required when Java cannot safely perform the conversion automatically.

Example

```
double a = 25.75;
```

```
int b = (int) a;
```

Here we manually tell Java:

```
(int) a
```

So:

```
double → int
```

```
25.75 → 25
```

The decimal part is removed.

This is called **explicit casting** or **narrowing casting**.

Types of Type Conversion/Casting

There are two important types:

1. Widening Conversion

Smaller type → Larger type

It is generally **automatic/implicit**.

Example:

```
int a = 10;
```

```
double b = a;
```

```
int → double
```

2. Narrowing Conversion

Larger type → Smaller type

It requires **explicit casting**.

Example:

```
double a = 10.5;
```

`int b = (int) a;`

`double → int`

Widening Conversion Example

Java can automatically convert:

`byte → short → int → long → float → double`

For example:

`int marks = 90;`

`double result = marks;`

Here `90` becomes `90.0` automatically.

Narrowing Casting Example

Suppose:

`double marks = 90.75;`

If we want an `int`:

`int result = (int) marks;`

Result:

`90`

The `.75` is lost.

Therefore, **narrowing conversion may cause data loss.**

Difference Between Type Conversion and Type Casting

Type Conversion	Type Casting
Usually automatic	Usually manual
Done by Java	Done by programmer
Also called implicit conversion	Also called explicit conversion
Commonly used for widening	Commonly used for narrowing
Example: <code>int → double</code>	Example: <code>double → int</code>
No casting operator required	Casting operator is required

Simple Example Showing Both

```
class Demo {  
    public static void main(String[] args) {  
        // Type Conversion  
        int a = 10;  
        double b = a;  
        // Type Casting  
        double x = 20.75;  
        int y = (int) x;  
        System.out.println(b);  
        System.out.println(y);  
    }  
}
```

Output:

10.0

20

★ Easy Trick to Remember

Conversion

👉 Java does it **automatically**

int → double

Casting

👉 Programmer does it **manually**

double → int

(int)

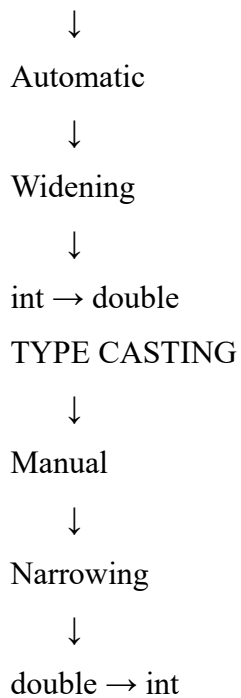
Exam Definition

Type Conversion: The automatic conversion of one data type into another compatible data type is called type conversion.

Type Casting: The explicit conversion of a value from one data type to another using a casting operator is called type casting.

In short:

TYPE CONVERSION



Scope of Variables

Scope means the area of a program where a variable can be accessed.

The **scope of a variable** means the **part of a program where a variable can be accessed or used**.

In simple words:

Scope tells us where a variable is available in a program.

For example, if a variable is declared inside a method, it can generally be used only inside that method.

Types of Variable Scope in Java

There are mainly **three important scopes**:

1. **Local Variable Scope**
2. **Instance Variable Scope**
3. **Static/Class Variable Scope**

There is also **block scope** for variables declared inside { }.

1. Local Variable

A variable declared **inside a method, constructor, or block** is called a local variable.

It can be accessed only within that particular method or block.

Example:

```
class Demo {  
    void display() {  
        int age = 20;  
        System.out.println(age);  
    }  
}
```

Here, **age** is a local variable.

It can be used inside **display()**:

```
System.out.println(age);
```

But it cannot be directly used outside **display()**.

Simple idea:

```
class  
└── display()  
    └── age
```

age is available only inside **display()**.

2. Block Scope

A variable declared inside a block **{ }** can be used **only within that block**.

Example:

```
class Demo {  
    public static void main(String[] args) {  
        if (true) {  
            int x = 10;  
            System.out.println(x);  
        }  
        // System.out.println(x); // Error
```

```
}  
}
```

Here, `x` is declared inside the `if` block.

Therefore, `x` is available only between:

```
{  
    int x = 10;  
}
```

Once the block ends, `x` cannot be accessed.

3. Instance Variable

A variable declared **inside a class but outside methods** is called an **instance variable**.

It belongs to an object of the class.

Example:

```
class Student {  
    int marks = 80;  
    void display() {  
        System.out.println(marks);  
    }  
}
```

Here, `marks` is an instance variable.

It can be accessed by the methods of the class through an object.

```
Student s = new Student();  
System.out.println(s.marks);
```

Scope

Its scope is generally the **class**, subject to access modifiers such as `private`, `public`, etc.

4. Static/Class Variable

A variable declared inside a class using the **static keyword** is called a static variable or class variable.

Example:

```

class Student {
    static String college = "ABC College";
    void display() {
        System.out.println(college);
    }
}

```

Here, `college` is a static variable.

It belongs to the **class rather than individual objects**.

It can be accessed using the class name:

```
System.out.println(Student.college);
```

Its scope is generally throughout the class, subject to its access modifier.

Difference Between Local, Instance and Static Variables

Variable	Declared	Scope	Example
Local	Inside method/block	Within that method/block	<code>int x = 10;</code>
Instance	Inside class, outside methods	Class/object context	<code>int marks = 80;</code>
Static	Inside class with <code>static</code>	Class-wide	<code>static int count = 0;</code>

Example of All Three

```

class Student {
    int marks = 80;          // Instance variable
    static String college = "ABC"; // Static variable
    void display() {
        int age = 20;        // Local variable
        System.out.println(marks);
        System.out.println(college);
        System.out.println(age);
    }
}

```

Here:

- `marks` → **Instance variable**
- `college` → **Static variable**

- age → **Local variable**

★ Easy Trick to Remember

Think about **where the variable is declared**:

Inside class + outside method

↓

Instance / Static Variable

Inside method

↓

Local Variable

Inside { }

↓

Block Variable

Exam Definition

The scope of a variable is the portion of a program in which that variable can be accessed or used. The scope depends on where the variable is declared.

Default Values of Variables

Instance and static variables are automatically given default values.

Data Type	Default Value
int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000'
boolean	false
Reference type	null

Important: Local variables do not get default values. They must be initialized before use.

LECTURE 4: Wrapper Classes, Comments and Garbage Collection

Wrapper Classes

1. What is a Wrapper Class?

In Java, **wrapper classes** are classes that convert primitive data types into objects.

Java has two types of data:

- **Primitive values** → `int`, `float`, `char`, `boolean`, etc.
- **Objects** → created from classes.

A wrapper class **wraps a primitive value inside an object**.

Easy Example

Suppose we have:

```
int age = 20;
```

Here, `age` is a **primitive value**.

If we want to store the same `20` as an **object**, we use the wrapper class:

```
Integer age = 20;
```

Here, `Integer` is a wrapper class for `int`.

Simple Meaning

Think of it like this:

Primitive value → put inside a class/object → **Wrapper Object**

`int 20` → `Integer` object containing `20`

2. Why Do We Need Wrapper Classes?

Why do we need an object when we already have a primitive value?

The main reason is:

Some Java features work only with objects.

For example, Java's **Collection Framework** such as `ArrayList` stores objects, not primitive data types.

We cannot write:

```
ArrayList<int> numbers;
```

Instead, we write:

```
ArrayList<Integer> numbers;
```

Example:

```
ArrayList<Integer> numbers = new ArrayList<>();  
numbers.add(10);  
numbers.add(20);  
numbers.add(30);
```

Here, `Integer` is used because `ArrayList` works with objects.

Easy way to remember

Primitive = simple value

Wrapper = object form of that value

3. Primitive Data Types and Their Wrapper Classes

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Important

Most wrapper class names start with a **capital letter**.

Example:

int → Integer
double → Double
char → Character
boolean → Boolean

4. Example of a Wrapper Class

```
public class Demo {  
    public static void main(String[] args) {  
        int a = 10;  
        Integer b = 20;  
        System.out.println(a);  
        System.out.println(b);  
    }  
}
```

Output

10

20

Here:

- `int a` → primitive variable
- `Integer b` → object/reference variable

5. Autoboxing

Java automatically converts a **primitive value into its corresponding wrapper object**.

This is called **Autoboxing**.

Example

```
int a = 10;  
Integer b = a;
```

Java automatically converts:

`int` → `Integer`

So we don't need to manually create the object.

Easy Definition

Autoboxing is the automatic conversion of a primitive value into its wrapper object.

Example

```
int marks = 90;  
Integer obj = marks;  
Here:
```

```
int marks = 90  
    ↓  
Integer obj = 90
```

6. Unboxing

Unboxing is the opposite of autoboxing.

It means converting a **wrapper object into a primitive value**.

Example

```
Integer obj = 50;  
int marks = obj;  
Java automatically converts:
```

Integer → int

This is called **Unboxing**.

Easy Definition

Unboxing is the automatic conversion of a wrapper object into its corresponding primitive value.

7. Autoboxing and Unboxing Together

```
public class Demo {  
    public static void main(String[] args) {  
        int a = 10;    // primitive  
        Integer b = a;  // Autoboxing  
        int c = b;     // Unboxing  
        System.out.println(c);  
    }  
}
```

Output

10

Flow

int

↓

Integer

↓

int

11. Useful Wrapper Class Methods

Some commonly used methods are:

Integer.parseInt()

Converts String into int.

```
int n = Integer.parseInt("25");
```

Double.parseDouble()

Converts String into double.

```
double n = Double.parseDouble("25.5");
```

Integer.toString()

Converts integer into String.

```
String s = Integer.toString(25);
```

Integer.max()

Finds the larger value.

```
int x = Integer.max(10, 20);
```

```
System.out.println(x);
```

Output:

20

12. Primitive vs Wrapper Class

Primitive	Wrapper
<code>int</code>	<code>Integer</code>
Stores a simple value	Stores value as an object
Faster and simpler	Provides useful methods
Cannot be used where an object is required	Can be used where objects are required
Example: <code>int x = 10;</code>	Example: <code>Integer x = 10;</code>

13. Very Easy Real-Life Example

Imagine you have a **book**.

The book itself is like a primitive value:

`int = 10`

Now suppose you want to put that book inside a **box**.

`Integer = [10]`

The box is like the **wrapper object**.

So:

Primitive value $\rightarrow$ Wrapper $\rightarrow$ Object

This is why it is called a **wrapper class**: it "wraps" the primitive value inside an object.

14. Important Exam Points

Students should remember these points:

1. **Wrapper classes convert primitive data types into objects.**
2. Every primitive data type has a corresponding wrapper class.
3. `int  $\rightarrow$  Integer`
4. `char  $\rightarrow$  Character`
5. `boolean  $\rightarrow$  Boolean`
6. **Autoboxing** = primitive $\rightarrow$ wrapper object.
7. **Unboxing** = wrapper object $\rightarrow$ primitive.
8. Wrapper classes are useful when Java requires an **object instead of a primitive value**.
9. Wrapper classes provide useful methods such as `parseInt()` and `parseDouble()`.

One-line definition for exam

Wrapper classes are predefined Java classes that represent primitive data types as objects.

Easy Memory Trick

Primitive		Wrapper
byte	→	Byte
short	→	Short
int	→	Integer
long	→	Long
float	→	Float
double	→	Double
char	→	Character
boolean	→	Boolean

Remember:

Autoboxing: `int` → `Integer`

Unboxing: `Integer` → `int`

Comment Syntax

Comments are used to **explain the code**. They are ignored by the compiler.

Single-Line Comment

```
// This is a comment
```

Multi-Line Comment

```
/*  
  
    This is a  
  
    multi-line comment  
  
*/
```

Documentation Comment

```
/**  
  
    * This is documentation.
```

Garbage Collection

Garbage Collection is the automatic process of removing objects that are no longer being used by a Java program.

Example:

```
Student s = new Student();  
  
s = null;
```

The `Student` object may now become eligible for garbage collection if no other reference points to it.

Advantages

- Automatically manages memory.
- Removes unused objects.
- Reduces memory management work for programmers.
- Helps prevent memory wastage.

Important: Garbage collection is performed by the **JVM**.

LECTURE 5: Arrays

Array

An array is a collection of **elements of the same data type** stored under a single variable name.

Example:

```
int marks[] = {70, 80, 90};
```

Array indexing starts from **0**.

```
marks[0] = 70
```

```
marks[1] = 80
```

```
marks[2] = 90
```

Declaration of Array

```
int marks[];
```

Creation of Array

```
marks = new int[3];
```

Initialization

```
marks[0] = 70;
```

```
marks[1] = 80;
```

```
marks[2] = 90;
```

It can also be written as:

```
int marks[] = {70, 80, 90};
```

Types of Arrays

One-Dimensional Array

Stores data in a single row.

```
int a[] = {10, 20, 30};
```

Two-Dimensional Array

Stores data in rows and columns.

```
int a[][] = {  
    {1, 2},  
    {3, 4}  
};
```

It can be represented as:

```
1  2
```

```
3  4
```

Multidimensional Array

An array having more than two dimensions.

```
int a[][][] = new int[2][2][2];
```

Jagged Array

1. What is a Jagged Array?

A **Jagged Array** is a **2D array in which each row can have a different number of elements**.

In a normal 2D array, every row usually has the same number of columns.

Normal 2D Array

10 20 30

40 50 60

70 80 90

Every row has **3 elements**.

Jagged Array

10 20

30 40 50

60

70 80 90 100

Here:

- Row 1 → 2 elements
- Row 2 → 3 elements
- Row 3 → 1 element
- Row 4 → 4 elements

So, **different rows have different sizes**.

2. Easy Definition

A **jagged array** is an array of arrays where each row can have a different number of elements.

3. Why is it called a Jagged Array?

Look at this:

10 20

30 40 50

60

70 80 90 100

The rows are not equal in length, so the right side looks **uneven or jagged**.

Therefore, it is called a **Jagged Array**.

LECTURE 6: String and StringBuffer

String

A **String** is a sequence of characters.

Example:

```
String name = "Java";
```

Creation of String

Using String Literal

```
String s = "Hello";
```

Using new

```
String s = new String("Hello");
```

String Concatenation

Concatenation means **joining two or more strings**.

The + operator can be used.

```
String a = "Hello";
```

```
String b = "Java";
```

```
String c = a + " " + b;
```

Output:

Hello Java

Changing Case

```
toUpperCase()
```

```
String s = "java";
```

```
System.out.println(s.toUpperCase());
```

Output:

JAVA

```
toLowerCase()
```

```
String s = "JAVA";
```

```
System.out.println(s.toLowerCase());
```

Output:

java

Character Extraction

The `charAt()` method is used to extract a character from a string.

```
String s = "Java";
```

```
System.out.println(s.charAt(0));
```

Output:

J

String Comparison

equals()

Compares the contents of two strings.

```
String a = "Java";
```

```
String b = "Java";
```

```
System.out.println(a.equals(b));
```

Output:

```
true
```

equalsIgnoreCase()

Compares two strings without considering case.

```
"Java".equalsIgnoreCase("JAVA");
```

Result:

```
true
```

StringBuffer

StringBuffer is a class used to create a **mutable sequence of characters**.

Example:

```
StringBuffer sb = new StringBuffer("Java");
```

```
sb.append(" Programming");
```

```
System.out.println(sb);
```

Output:

```
Java Programming
```

Method	Use
<code>append()</code>	Adds text
<code>insert()</code>	Inserts text
<code>delete()</code>	Deletes characters
<code>reverse()</code>	Reverses the string

LECTURE 7: Operators and Mathematical Functions

Operators are symbols used to perform operations on values and variables.

Arithmetic Operators

Operator	Meaning
<code>+</code>	Addition
<code>-</code>	Subtraction
<code>*</code>	Multiplication
<code>/</code>	Division
<code>%</code>	Remainder

Example:

```
int a = 10;
```

```
int b = 3;
```

```
System.out.println(a + b);
```

```
System.out.println(a % b);
```

Relational Operators

Relational operators compare two values.

Operator	Meaning
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal
<code><=</code>	Less than or equal

Result is always `true` or `false`.

Logical Operators

Operator Meaning

&& AND

、

! NOT

Example:

```
if(age >= 18 && age <= 60)
    System.out.println("Valid");
```

Bitwise Operators

Bitwise operators work at the **bit level**.

Operator Meaning

& Bitwise AND

、

、

^ Bitwise XOR

~ Bitwise complement

<< Left shift

>> Right shift

Assignment Operators

Operator Example

= a = 10

+= a += 5

-= a -= 5

*= a *= 5

/= a /= 5

%= a %= 5

Increment and Decrement

Increment ++

Increases value by 1.

```
a++;
```

Decrement --

Decreases value by 1.

```
a--;
```

They can be used as **prefix** or **postfix** operators.

```
++a;
```

```
a++;
```

Conditional / Ternary Operator

The `?:` operator is called the **conditional or ternary operator**.

Syntax

```
condition ? value1 : value2;
```

Example:

```
int a = 10;
```

```
int b = 20;
```

```
int max = (a > b) ? a : b;
```

Mathematical Functions

Java provides mathematical functions through the `Math` class.

Examples:

```
Math.sqrt(25);        // 5.0
```

```
Math.pow(2, 3);        // 8.0
```

```
Math.abs(-10);         // 10
```

```
Math.max(10, 20);      // 20
```

```
Math.min(10, 20);      // 10
```

LECTURE 8: Decision and Control Statements

Control statements control the **order in which statements are executed**.

They are mainly divided into:

1. Selection statements
2. Looping statements
3. Jump statements

1. **if** Statement

The **if** statement executes a block only when the condition is true.

Syntax

```
if(condition) {  
    statements;  
}
```

Example:

```
if(age >= 18) {  
    System.out.println("Eligible");  
}
```

2. **if-else** Statement

Used when there are two possible conditions.

```
if(age >= 18) {  
    System.out.println("Adult");  
}  
else {  
    System.out.println("Minor");  
}
```

3. switch Statement

The switch statement is used when one expression has multiple possible values.

```
int day = 1;

switch(day) {
    case 1:
        System.out.println("Monday");
        break;

    case 2:
        System.out.println("Tuesday");
        break;

    default:
        System.out.println("Invalid");
}
```

Important Keywords

- **case** – defines a possible value.
- **break** – terminates the switch.
- **default** – executes when no case matches.

Loops

Loops are used to **execute a block of statements repeatedly**.

4. while Loop

The condition is checked **before** executing the loop body.

```
int i = 1;
```

```
while(i <= 5) {  
    System.out.println(i);  
    i++;  
}
```

5. do-while Loop

The loop body executes **at least once**, because the condition is checked after the body.

```
int i = 1;  
  
do {  
    System.out.println(i);  
    i++;  
} while(i <= 5);
```

6. for Loop

The for loop is commonly used when the number of repetitions is known.

```
for(int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Structure

initialization → condition → body → increment/decrement

7. break

The `break` statement immediately terminates a loop or switch.

```
for(int i = 1; i <= 5; i++) {  
    if(i == 3)  
        break;  
  
    System.out.println(i);  
}
```

Output:

1

2

8. continue

The `continue` statement skips the current iteration and moves to the next iteration.

```
for(int i = 1; i <= 5; i++) {  
    if(i == 3)  
        continue;  
  
    System.out.println(i);  
}
```

Output:

1

2

4

5

9. return

The `return` statement is used to **return control from a method**.

Example:

```
return;
```

It can also return a value:

```
return a + b;
```

10. System.exit()

`System.exit()` is used to **terminate the Java program**.

```
System.exit(0);
```

0 generally indicates normal termination.
